
Overparameterized Models

Rajat Narandekar
rajatn@iisc.ac.in
Indian Institute of Science

Anmol Mittal
anmolmittal@iisc.ac.in
Indian Institute of Science

Rajul Mahawar
rajulm@iisc.ac.in
Indian Institute of Science

Palak Raisinghani
palakr@iisc.ac.in
Indian Institute of Science

Anmol Agarwal
anmola@iisc.ac.in
Indian Institute of Science

1 Introduction to OPM

Overparameterized models are heavily employed in real life applications for performing crucial tasks in domains like computer vision, NLP and various others. These models are characterized by having significantly more parameters than the number of training samples. This leads to them being able to achieving near zero training error. Historically, the classical statistical viewpoint cautioned against such architectures, as traditional theory suggested that fitting more parameters than data points would lead to severe overfitting and poor generalization. Back then the practice in the under parameterized regime was to navigate the bias-variance tradeoff, aiming for a "sweet spot" between underfitting and interpolation to minimize expected risk on unseen data. Consequently, OPMs were theoretically expected to perform poorly.

Emperical results however challenged the classical viewpoint. The graphs of test error against model size observed a second descent after the interpolating region - a phenomenon now referred to as **Double descent**. Many explanations of generalization error that worked well in the classical regime to explain model behavior, turn out to be vacuously true in the over parameterized region and do not provide much insight on the actual performance of the model. This called for the need to **rethink generalization** in the context of over parameterized models and find better and more insightful tighter bounds.

The report aims to uncover the nature of over parameterized models. How they learn data and how their performance is affected by changes in regularization methods, choice of activation functions and optimizer algorithms. The experiments and implementations corresponding to this study are available at:

<https://github.com/Pal-R-S/OverParameterizedModels>

2 Learning characteristics

Over parameterized models fit perfectly to training data. Classically this was believed to be akin to interpolation and hence it was the popular viewpoint that won't generalize well. Understanding the learning characteristic of these models sheds light on their ability to generalize well.

Label Randomization Experiment

Refer to 7.1 for details on the experimental setup

Observations and Results: We train a model from scratch on several values of p . even with complete label randomization ($p=1$), the model attains nearly zero training error with its training curve not varying greatly from the case of no label randomization($p=0$). This shares the key insight that OPMs easily fit random labels. Training time taken to converge to zero training error for $p=1$ case is more than that for $p=0$ only by a small constant factor. This shows that label randomization works solely as a data transformation leaving all other characteristics of the learning problem unchanged.

While all the training errors converge to zero in figure 1b, the test errors for different values of p converge to different values varying gradually with value of p . We take a finer look at the test

error for $p=5$, though a similar analysis can be done for other values of p and will lead to the same conclusions. Please refer to the formulation of the Bayes’ predictor and simple interpolator given in Appendix 7.1 for the calculations of the numerical values of risk of the Bayes’ predictor Bayes’ : 0.45; Simple Interpolator : 0.675, Observed test error : ≈ 0.51 as seen in 1a.

The observed test error of the OPM (≈ 0.51) as seen in 1c is much closer to the the Bayes’ optimal performance than the simple interpolator. This is evidence that the Over parameterized model isn’t a simple interpolation and learns the signal present in the data. This phenomenon is better explained in the context of benign overfitting but we exclude that from our discussion for now.

We conclude this section with the observation that learning a smoother global function and locally fitting to noise leads to better generalization than simple interpolation of data points. OPMs perform well since they don’t act as simple interpolator but rather learns the signal in the data and fits the noise.

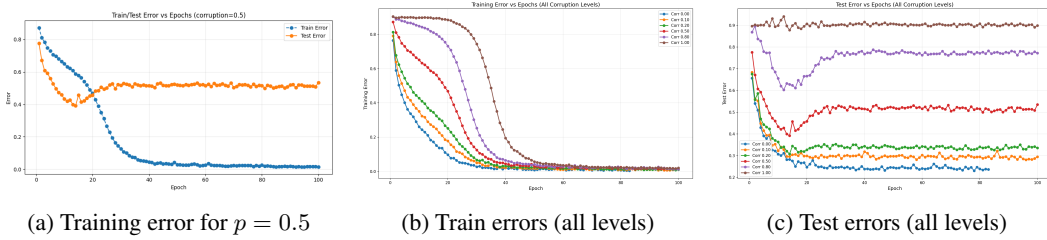


Figure 1: CIFAR-10 Label Randomization Experiments: Comparison of training and test error across varying levels of label noise.

3 Double Descent

We study the double descent phenomenon across different model classes to understand how model complexity affects generalization. In particular, we consider: (i) neural networks on MNIST, (ii) linear regression with Gaussian inputs, and (iii) Gaussian projection models which better reflect practical behavior.

Observations and Results:

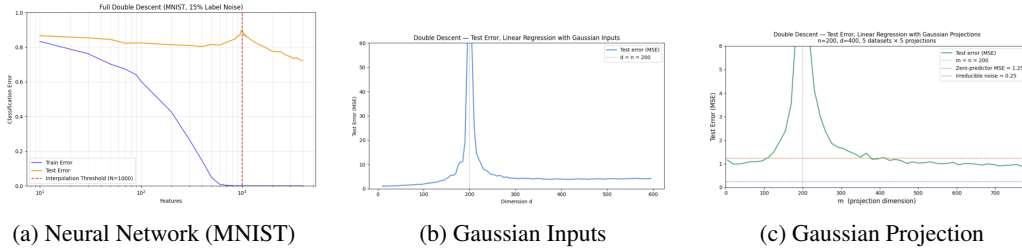


Figure 2: Comparison of double descent across different models. Left: neural network shows clear second descent. Middle: Gaussian inputs exhibit a sharp peak near interpolation. Right: Gaussian projection recovers a more realistic double descent behavior.

As model complexity (number of parameters P) increases, the training error decreases monotonically and reaches nearly zero around the interpolation threshold ($P \approx n$). Refer to Appendix for details on the experimental setup.

The test error initially decreases, then sharply increases near interpolation, and finally decreases again in the overparameterized regime. This confirms the characteristic *double descent* behavior. The second descent suggests that highly overparameterized neural networks generalize well despite perfectly fitting the training data.

Gaussian Input Model

We also study linear regression with Gaussian inputs, where complexity is controlled by the input dimension d .

While the test error exhibits a sharp peak near the interpolation threshold ($d \approx n$), the second descent is weak or absent. This is because the model is too simple and lacks the expressive power and implicit regularization present in neural networks. The behavior is largely dominated by variance.

Gaussian Projection Model

To bridge the gap between simple linear models and neural networks, we consider Gaussian projections. In this setup, inputs are projected onto a lower-dimensional random subspace before performing regression.

We observe a clear double descent curve: the test error peaks near the interpolation threshold ($m \approx n$) and decreases again in the overparameterized regime. Compared to the plain Gaussian input model, this setting exhibits a more realistic second descent.

This demonstrates that incorporating random feature representations introduces behavior closer to modern machine learning models.

Conclusion:

We observe that neural networks exhibit a clear double descent behavior, which can be attributed to their high expressive power and the implicit regularization induced by the optimization process. In contrast, simple Gaussian linear models capture only the interpolation peak and fail to demonstrate a pronounced second descent. The Gaussian projection model serves as an intermediate setting, where the introduction of random feature representations leads to the reappearance of double descent behavior.

This suggests that model expressivity and feature representation play a crucial role in understanding generalization beyond classical bias-variance trade-offs.

4 Is Explicit Regularization the secret?

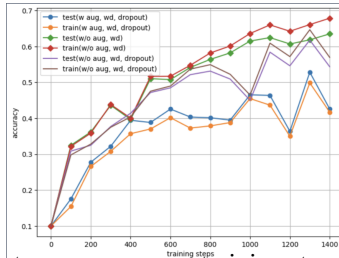


Figure 3: Training and test accuracy versus training steps for ResNet18 on CIFAR-10 under three settings: (i) data augmentation, weight decay, and dropout; (ii) weight decay only; and (iii) no explicit regularization. Models are trained using SGD with momentum (0.9) and learning rate 0.1 for 1500 steps. Accuracy is evaluated periodically on training and test sets with fixed data splits and random seeds.

The results of our experiments indicate that all models, regardless of the use of explicit regularization, achieve steadily increasing training accuracy, reaching high values as training progresses. This shows that modern neural networks have sufficient capacity to fit the training data even in the absence of explicit regularization. While techniques such as data augmentation, weight decay, and dropout provide slight improvements in test accuracy and help stabilize training, the overall differences between regularized and unregularized models remain modest.

These observations suggest that explicit regularization is neither necessary nor sufficient to explain generalization in overparameterized models. All configurations operate in a regime where the model is capable of interpolating the training data, and generalization is influenced more by implicit biases of the optimization process than by explicit regularization techniques. Thus, while explicit regularization may yield quantitative improvements, it does not fundamentally account for the strong generalization performance observed in modern deep neural networks.

5 Implicit Bias of SGD

Modern overparameterized models admit many solutions that perfectly fit the training data, yet optimization algorithms consistently select specific ones. This phenomenon, known as *implicit bias*, plays a central role in understanding generalization. In this section, we empirically study the implicit bias of SGD in both regression and classification settings, and compare its behavior with other optimizers.

5.1 Bias of SGD in Linear Regression

In noiseless linear regression, it is known that SGD in the overparameterized regime converges to the minimum norm solution satisfying $Xw = y$. To verify this by performing an experiment, we vary the model dimension d to explore underparameterized, critically parameterized, and overparameterized regimes, and train using SGD on the squared loss $\|X_d w - y\|^2$.

We track the parameter norm $\|w_t\|_2$ over iterations. In the overparameterized case ($d > n$), there are infinitely many solutions to $X_d w = y$, and the minimum norm solution is $w^* = X_d^\top (X_d X_d^\top)^{-1} y$. For more details refer to 7.5.1

Figure 4b shows that SGD converges to this solution, confirming its implicit bias toward minimum norm solutions in overparameterized linear regression.

5.2 Bias of SGD in Classification

For linearly separable data, the norm of the solution w_t found by SGD can grow unbounded, but its direction converges to the maximum margin solution.

We verify this by performing an experiment with using a synthetic dataset with $X \in \mathbb{R}^{n \times d}$, where $n = 100, d = 20$, sampled from $\mathcal{N}(0, I)$, and ground truth $w^* \sim \mathcal{N}(0, I)$. Labels are generated as $y = \text{sign}(Xw^*)$, on Logistic loss

From Figure 4a, as model size increases, the margin grows and converges to a value consistent with max-margin behavior. For more details refer to 7.5.2.

5.3 Is it just SGD?

We compare SGD with Adam and SGD with momentum.

From Figure 4c, Adam reaches a separating solution faster but does not converge to the maximum margin classifier. Momentum accelerates early progress compared to SGD, achieving positive margin sooner. However, vanilla SGD continues improving and ultimately achieves a larger margin.

Overall, SGD shows a clear bias toward the maximum margin solution, unlike the other optimizers.

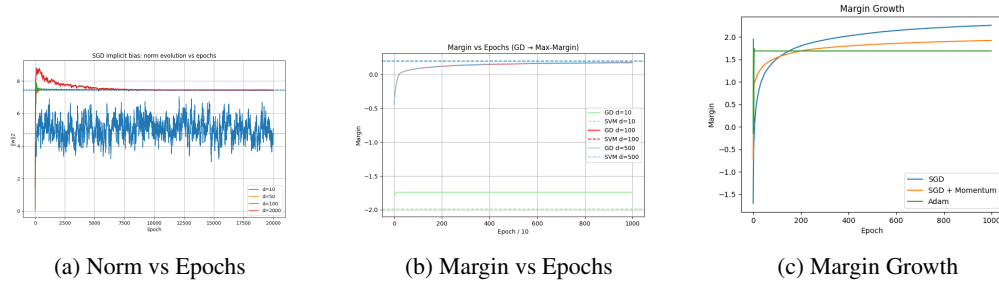


Figure 4: Implicit bias experiments

6 Effect of Activation Functions on Generalization

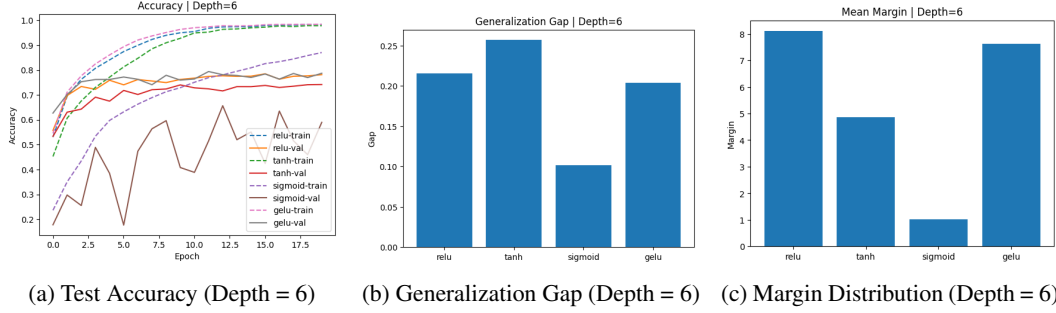


Figure 5: Comparison of test accuracy, generalization gap, and margin distribution at depth 6 across activation functions on CIFAR-10 using a CNN architecture.

We investigate how different activation functions (ReLU, GELU, Tanh, and Sigmoid) influence generalization, not by altering representational capacity, but through their impact on optimization dynamics and gradient propagation.

Empirically, we observe that ReLU and GELU consistently achieve high training, validation, and test accuracy across varying depths, while tanh exhibits slower convergence and moderate degradation with depth. Sigmoid performs poorly, often failing to train effectively, particularly in deeper networks. As shown in Figure 5, ReLU and GELU achieve the highest test accuracy at depth 6, while Sigmoid significantly underperforms. Additionally, although Sigmoid shows a small generalization gap, this is due to underfitting, as both training and test accuracies remain low. In contrast, ReLU and GELU exhibit a moderate gap but achieve significantly better test performance, highlighting that the generalization gap alone is not a reliable indicator of model quality (Figure 5).

Margin-based analysis further supports these findings. As illustrated in Figure 5, ReLU and GELU produce margin distributions that are shifted toward larger positive values, indicating strong class separation and high confidence. Tanh achieves moderate margins, while Sigmoid margins remain concentrated near zero, reflecting poor feature learning. Moreover, increasing depth improves margins for ReLU and GELU, has limited effect for Tanh, and provides negligible benefit for Sigmoid.

These observations can be explained by differences in gradient propagation. Sigmoid suffers from severe vanishing gradients due to bounded derivatives, leading to ineffective optimization. Tanh partially mitigates this issue but still experiences gradient decay in saturated regions. In contrast, ReLU and GELU preserve gradient flow, enabling efficient optimization and the learning of discriminative representations.

In conclusion, activation functions influence generalization indirectly by shaping optimization dynamics rather than just model capacity. Stable gradient propagation leads to better optimization, improved margin formation, and ultimately stronger generalization.

References

- [1] Francis Bach. *Learning Theory from First Principles*. MIT Press, 2023.
- [2] Simon S. Du, Xiyu Zhai, Barnabas Poczos, Aarti Singh, and Andrea Montanari. Gradient descent provably optimizes over-parameterized neural networks. *International Conference on Learning Representations (ICLR)*, 2019.
- [3] Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. *arXiv preprint arXiv:1912.02292*, 2019.
- [4] Gal Vardi. On the implicit bias in deep-learning algorithms. *Communications of the ACM*, May 2023. Research and Advances section, Posted May 17, 2023.
- [5] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.

7 Appendix

7.1 Bayes predictor and Interpolator- expected performance

We consider a C -class classification problem where both the training and test sets undergo label randomization to an extent $p \in [0, 1]$. We define "Signal" as the event where a sample retains its true ground-truth label, and "Noise" as the event where the label is replaced by a uniform random draw from all C classes.

Experimental details: 40% of the CIFAR10 dataset. We randomize levels to p percent randomization. Randomization is done as follows: if $np.rand() \geq 1 - p$ we uniformly select 1 of the 10 classes as the new label, otherwise the true label is used as the new label. The training is run on the training split and evaluated on the test split at each epoch.

1. Bayes' Predictor Performance

The Bayes' optimal predictor $h^*(x)$ always predicts the most probable class (the true signal), regardless of the noise it was exposed to during training. Its performance depends solely on the probability that the *test* label matches the true signal.

Table 1: Expected Accuracy: Bayes' Predictor

Event (Test Label)	Probability	$P(\hat{y} = y_{test})$	Contribution
Is True Signal	$1 - p + \frac{p}{C}$	1	$1 - p + \frac{p}{C}$
Is Random ($\neq$ Signal)	$\frac{p(C-1)}{C}$	0	0

Summary Results for Bayes' Predictor:

- **Test Accuracy:** $1 - p + \frac{p}{C}$
- **Test Error:** $p \left(\frac{C-1}{C} \right)$

2. Simple Interpolator Performance

A simple interpolator behaves like a 1-Nearest Neighbor classifier. Its prediction $\hat{y}$ is "Signal" with probability $(1 - p)$ and "Noise" with probability p . The test accuracy is the probability that the interpolated label matches the noisy test label.

Table 2: Expected Accuracy: Simple Interpolation (1-NN)

Event Configuration	Probability	$P(\hat{y} = y_{test})$	Contribution
Both are Signal	$(1 - p)^2$	1	$(1 - p)^2$
1-NN Noise, Test Signal	$p(1 - p)$	$1/C$	$\frac{p(1-p)}{C}$
1-NN Signal, Test Noise	$(1 - p)p$	$1/C$	$\frac{p(1-p)}{C}$
Both are Noise	p^2	$1/C$	$\frac{p^2}{C}$

Summary Results for Interpolator:

- **Test Accuracy:** $(1 - p)^2 + \frac{2p(1-p)+p^2}{C} = (1 - p)^2 + \frac{2p-p^2}{C}$
- **Test Error:** $1 - \left[(1 - p)^2 + \frac{2p-p^2}{C} \right] = (2p - p^2) \left(\frac{C-1}{C} \right)$

7.2 Experimental Setup in Double Descent

Dataset

We use the MNIST dataset and construct a binary classification task by selecting two digit classes. Labels are mapped to $\{0, 1\}$.

To study robustness, label noise is introduced by randomly flipping a fixed fraction (e.g., 15%) of training labels.

Model Architecture

We use a fully connected neural network with:

- Input dimension: d
- One hidden layer with H units
- Output dimension: $K = 2$

The total number of parameters is:

$$P = (d + 1)H + (H + 1)K.$$

By varying H , we control model complexity.

Training Objective

The model is trained using mean squared error (MSE):

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n \|f_{\theta}(x_i) - y_i\|^2.$$

Training Procedure

For each choice of H (hence P):

1. Initialize model parameters randomly,
2. Train using gradient-based optimization until convergence,
3. Record training error,
4. Evaluate test error on held-out data.

Evaluation Metrics

- Training Error: fraction of misclassified training samples
- Test Error: fraction of misclassified test samples

7.3 Gaussian Input Experiment

7.3.1 Data Generation

We generate a synthetic dataset with $n = 200$ samples. Each input vector $x \in \mathbb{R}^d$ is drawn independently from a standard Gaussian distribution:

$$x \sim \mathcal{N}(0, I_d).$$

The output is generated according to a linear model with additive Gaussian noise:

$$y = x^{\top} \theta^* + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2),$$

where $\theta^* \in \mathbb{R}^d$ is a fixed ground-truth parameter vector and $\sigma = 1.0$.

7.3.2 Model and Estimation

For each dimension d , we fit a linear model using the minimum-norm least squares solution:

$$\hat{\theta} = \arg \min_{\theta} \|X\theta - y\|_2^2,$$

which is computed via the Moore–Penrose pseudoinverse:

$$\hat{\theta} = X^{\dagger} y.$$

7.3.3 Varying Model Complexity

Model complexity is controlled by varying the input dimension d . We sweep:

$$d \in \{10, 20, \dots, 600\}.$$

This allows us to study the transition across:

- Underparameterized regime ($d < n$),
- Interpolation threshold ($d \approx n$),
- Overparameterized regime ($d > n$).

7.3.4 Evaluation

For each value of d , we:

1. Generate a training dataset of size n ,
2. Compute $\hat{\theta}$,
3. Evaluate test error on an independently sampled test set.

The test error is computed as:

$$\frac{1}{n} \sum_{i=1}^n (y_i^{\text{test}} - x_i^{\text{test}\top} \hat{\theta})^2.$$

7.4 Gaussian Projection Model

To obtain a more realistic setting, we introduce Gaussian projections. Each input $x_i \in \mathbb{R}^D$ is first drawn as:

$$x_i \sim \mathcal{N}(0, I_D).$$

We then project x_i onto an m -dimensional subspace using a random Gaussian matrix $W \in \mathbb{R}^{m \times D}$:

$$z_i = \frac{1}{\sqrt{m}} W x_i, \quad W_{jk} \sim \mathcal{N}(0, 1).$$

The regression model is trained on the transformed features z_i , with outputs generated as:

$$y_i = z_i^\top \theta^* + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Here, model complexity is controlled by the projection dimension m . By varying m , we observe behavior across underparameterized and overparameterized regimes, with interpolation occurring near $m \approx n$. For a detailed theoretical treatment of this phenomenon, refer to Chapter 12, Sections 12.2.3 and 12.2.4 of Bach's book:

https://www.di.ens.fr/~fbach/ltpf_book.pdf

7.5 Bias of SGD in Over-Parameterized Models

7.5.1 Bias in Linear Regression

Minimum Norm Interpolation via Constrained Problem

Consider the constrained optimization problem:

$$\min_w \frac{1}{2} \|w\|^2 \quad \text{subject to } Xw = y$$

Lagrangian:

$$\mathcal{L}(w, \alpha) = \frac{1}{2} \|w\|^2 + \alpha^\top (y - Xw)$$

Stationarity condition:

$$\nabla_w \mathcal{L} = w - X^\top \alpha = 0 \Rightarrow w = X^\top \alpha$$

Substituting into the Lagrangian:

$$\mathcal{L}(\alpha) = \frac{1}{2} \|X^\top \alpha\|^2 + \alpha^\top (y - XX^\top \alpha)$$

Expanding the norm:

$$\|X^\top \alpha\|^2 = \alpha^\top XX^\top \alpha$$

Thus,

$$\mathcal{L}(\alpha) = \frac{1}{2} \alpha^\top XX^\top \alpha + \alpha^\top y - \alpha^\top XX^\top \alpha = \alpha^\top y - \frac{1}{2} \alpha^\top XX^\top \alpha$$

Dual problem:

$$\max_{\alpha} \alpha^\top y - \frac{1}{2} \alpha^\top X X^\top \alpha$$

Optimality condition:

$$\nabla_{\alpha} = y - X X^\top \alpha = 0 \Rightarrow X X^\top \alpha = y$$

$$\alpha^* = (X X^\top)^{-1} y$$

Primal solution:

$$w^* = X^\top \alpha^* = X^\top (X X^\top)^{-1} y$$

Solution via Gradient Descent / SGD Dynamics

Consider the empirical risk minimization problem:

$$\min_{w \in \mathbb{R}^d} \frac{1}{2n} \|Xw - y\|_2^2$$

Gradient:

$$\nabla L(w) = \frac{1}{n} X^\top (Xw - y)$$

Gradient descent updates:

$$w_{t+1} = w_t - \eta X^\top (Xw_t - y)$$

Span preservation. We prove by induction that if $w_0 = 0$, then $w_t \in \text{span}(X^\top)$ for all t .

Base case:

$$w_0 = 0 \in \text{span}(X^\top)$$

Inductive step: assume $w_t \in \text{span}(X^\top)$, so $w_t = X^\top \beta_t$ for some β_t . Then

$$\begin{aligned} Xw_t &= X X^\top \beta_t \\ X^\top (Xw_t - y) &= X^\top (X X^\top \beta_t - y) \in \text{span}(X^\top) \\ w_{t+1} &= w_t - \eta X^\top (Xw_t - y) \in \text{span}(X^\top) \end{aligned}$$

Thus, for all t ,

$$w_t \in \text{span}(X^\top)$$

Taking limits,

$$w^* \in \text{span}(X^\top) \Rightarrow w^* = X^\top \alpha$$

Characterization of the limit point.

At convergence:

$$\nabla L(w^*) = 0 \Rightarrow X^\top (Xw^* - y) = 0$$

Substituting $w^* = X^\top \alpha$:

$$\begin{aligned} X^\top (X X^\top \alpha - y) &= 0 \\ X^\top X X^\top \alpha &= X^\top y \end{aligned}$$

Since both sides lie in $\text{span}(X^\top)$, it follows that

$$\begin{aligned} X X^\top \alpha &= y \\ \alpha &= (X X^\top)^{-1} y \end{aligned}$$

Hence,

$$w^* = X^\top (X X^\top)^{-1} y$$

Equivalence of the two approaches

Both the constrained minimum-norm problem

$$\min \|w\|^2 \quad \text{subject to } Xw = y$$

and gradient descent (or SGD) initialized at $w_0 = 0$ lead to the same solution:

$$w^\star = X^\top (XX^\top)^{-1}y$$

This demonstrates that gradient-based methods implicitly select the minimum-norm interpolating solution in over-parameterized linear regression.

7.5.2 Equivalence of Logistic Loss Minimization and Hard-Margin SVM (Separable Case)

Assume a dataset $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{-1, 1\}$ that is linearly separable.

We consider the logistic loss:

$$\ell(w) = \sum_{i=1}^n \log(1 + e^{-y_i w^\top x_i})$$

Step 1: No Finite Minimizer

Since the data is separable, there exists w_0 such that:

$$y_i w_0^\top x_i > 0 \quad \forall i$$

For any $\alpha > 0$, let $w = \alpha w_0$. Then:

$$\ell(\alpha w_0) = \sum_{i=1}^n \log(1 + e^{-\alpha y_i w_0^\top x_i})$$

As $\alpha \rightarrow \infty$:

$$e^{-\alpha y_i w_0^\top x_i} \rightarrow 0 \Rightarrow \ell(\alpha w_0) \rightarrow 0$$

Thus, the infimum of the loss is 0, but it is not attained at any finite w .

Step 2: Directional Decomposition

Write:

$$w = \alpha \hat{w}, \quad \|\hat{w}\| = 1$$

Then:

$$\ell(w) = \sum_{i=1}^n \log(1 + e^{-\alpha y_i \hat{w}^\top x_i})$$

For large α , we use $\log(1 + e^{-z}) \approx e^{-z}$ when $z \rightarrow \infty$, hence:

$$\ell(w) \approx \sum_{i=1}^n e^{-\alpha y_i \hat{w}^\top x_i}$$

Step 3: Dominance of Minimum Margin

Define the margin:

$$\gamma(\hat{w}) = \min_i y_i \hat{w}^\top x_i$$

Then the largest term in the sum dominates:

$$\ell(w) \approx e^{-\alpha \gamma(\hat{w})}$$

Step 4: Optimization over Direction

To minimize $\ell(w)$ as $\alpha \rightarrow \infty$, we must maximize $\gamma(\hat{w})$:

$$\max_{\|\hat{w}\|=1} \min_i y_i \hat{w}^\top x_i$$

Step 5: Reduction to Hard-Margin SVM

Rescale w so that:

$$\min_i y_i w^\top x_i = 1$$

Then maximizing margin is equivalent to:

$$\max \frac{1}{\|w\|} \iff \min \|w\|^2$$

Thus, we obtain the hard-margin SVM problem:

$$\min_w \frac{1}{2} \|w\|^2 \quad \text{such that } y_i w^\top x_i \geq 1 \quad \forall i$$

Conclusion

As $\|w\| \rightarrow \infty$, minimizing logistic loss selects the direction:

$$w^* = \arg \max_{\|w\|=1} \min_i y_i w^\top x_i$$

which is exactly the maximum-margin separator.

Logistic loss minimization on separable data converges in direction to the hard-margin SVM solution

7.6 Effect of Activation functions on Generalization Gap

7.6.1 Mathematical Analysis of Gradient Propagation

Consider a deep network of depth L :

$$h_L = f_L(f_{L-1}(\cdots f_1(x)))$$

Using backpropagation:

$$\frac{\partial \mathcal{L}}{\partial x_0} = \prod_{l=1}^L \frac{\partial x_l}{\partial x_{l-1}} \cdot \frac{\partial \mathcal{L}}{\partial x_L}$$

Each term contains activation derivatives:

$$\frac{\partial x_l}{\partial x_{l-1}} = W_l \cdot f'_l(x_{l-1})$$

Thus, gradient magnitude is governed by:

$$\left\| \frac{\partial \mathcal{L}}{\partial x_0} \right\| \propto \prod_{l=1}^L \|f'_l(x)\|$$

Sigmoid: Exponential Decay

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$$

$$\Rightarrow \left\| \frac{\partial \mathcal{L}}{\partial x_0} \right\| \leq (0.25)^L$$

This leads to exponential vanishing gradients.

Tanh: Partial Decay

$$\tanh'(x) = 1 - \tanh^2(x)$$

which attains its maximum value of 1 at $x = 0$. For inputs near zero (a common regime during training due to normalization and initialization), $\tanh'(x) \approx 1$, so gradients are largely preserved across layers. Only in the saturated regime ($|x| \gg 1$) does $\tanh'(x) \rightarrow 0$, causing decay.

Therefore, unlike sigmoid's uniform exponential contraction, tanh exhibits *data-dependent* gradient decay, which is typically slower because many activations operate in the near-linear region where derivatives remain close to 1.

ReLU: Gradient Preservation

$$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

For active neurons:

$$\prod_{l=1}^L f'_l(x) \approx 1$$

GELU: Smooth Gradient Flow

$$\text{GELU}(x) = x \cdot \Phi(x)$$

Derivative:

$$\text{GELU}'(x) = \Phi(x) + x \cdot \phi(x)$$

where $\phi(x)$ is the Gaussian PDF.

This ensures:

- Non-zero gradients near zero
- Smooth transitions (better optimization landscape)

7.6.2 Generalization Gap Analysis

The generalization gap is defined as:

$$\text{Gap} = A_{\text{train}} - A_{\text{test}}$$

We decompose test error:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}^2}_{\text{underfitting}} + \underbrace{\text{Variance}}_{\text{overfitting}} + \underbrace{\text{Noise}}_{\text{irreducible}}$$

Interpretation Across Activations

- Sigmoid: High bias $\Rightarrow$ low gap but poor performance
- ReLU/GELU: Balanced bias-variance $\Rightarrow$ moderate gap, high accuracy
- Tanh: Intermediate regime

7.6.3 Margin Theory and Generalization

Margin for a sample (x, y) :

$$\gamma(x, y) = f_y(x) - \max_{j \neq y} f_j(x)$$

7.6.4 Depth Scaling Behavior

We analyze how depth L affects gradients:

$$\prod_{l=1}^L f'_l(x)$$

- Sigmoid: $(0.25)^L \rightarrow 0$
- Tanh: $\rightarrow 0$ slowly
- ReLU/GELU: ≈ 1

Thus, effective depth is activation-dependent.

7.6.5 Link to Implicit Bias

Optimization induces implicit bias:

This explains why margin correlates strongly with test performance. Mean margin:

$$\bar{\gamma} = \frac{1}{N} \sum_{i=1}^N \gamma(x_i, y_i)$$

Connection to Generalization Bounds

From margin-based theory:

$$\mathcal{R}(f) \leq \hat{\mathcal{R}}_{\gamma}(f) + \mathcal{O}\left(\frac{1}{\gamma\sqrt{n}}\right)$$

Thus:

- Larger margins $\Rightarrow$ tighter generalization bounds
- Smaller margins $\Rightarrow$ weaker guarantees

7.6.6 Complete Set of Experimental Plots

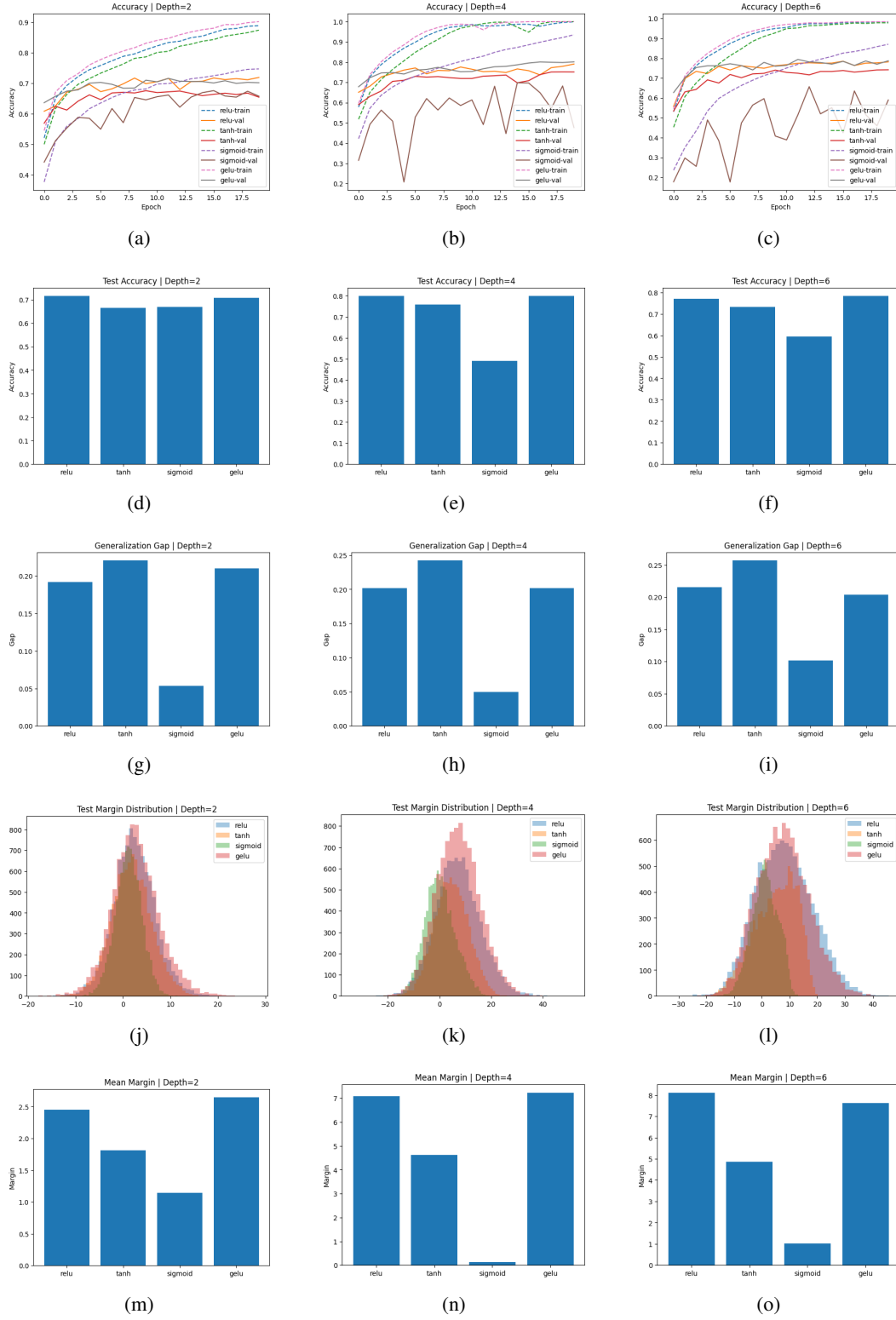


Figure 6: Complete set of 15 experimental plots across depths and activation functions. For Margin distribution plots ((j), (k), (l)) across depths and activation functions. The x-axis represents the margin (γ), and the y-axis represents the number of samples.